Correction - Quizz 6 - Graphes

Durée: 30min **2025/2026**

L'utilisation de toute calculatrice, de tout matériel électronique et de tout document est interdite

1. Question de cours

Q1. **/1.5**

Répondre aux questions suivantes sur le graphe 1 :

- Orienté : □ Oui X Non
- Pondéré : □ Oui X Non
- Connexe : □ Oui X Non
- Degré du sommet D : 3
- Nombre de composantes connexes : 2
- Nombre de cycles : 1

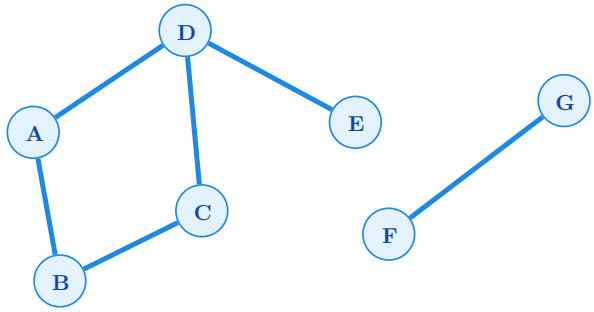

Fig. 1. – Graphe 1

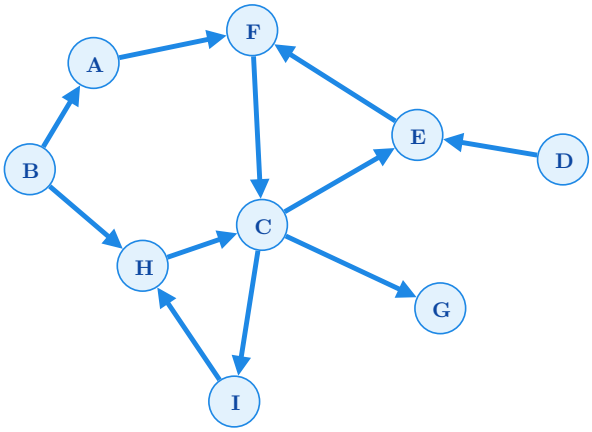

Fig. 2. – Graphe 2

Q2. **/1.5**

Répondre aux questions suivantes sur le graphe 2 :

- Orienté : X Oui □ Non
- Pondéré : □ Oui X Non
- Fortement connexe : □ Oui X Non
- Degrés du sommet C : $d^+(C) = 2$ et $d^-(C) = 3$
- Nombre de cycles : 2
- F est-il accessible depuis I ? X Oui □ Non
- A est-il accessible depuis I ? □ Oui X Non

2. Représentations et fonctions de base

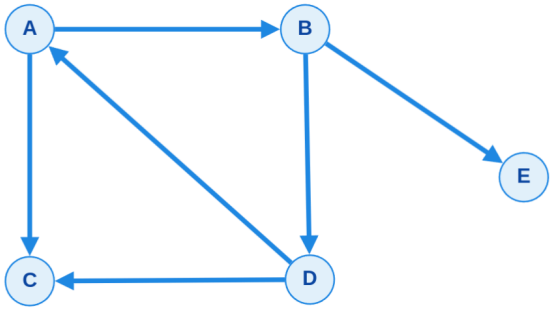

Fig. 3. – Graphe 3

Q3. **/1**

Donnez la matrice d'adjacence du **Graphe 3**.

```
m = [
  [0,1,1,0,0],
  [0,0,0,1,1],
  [0,0,0,0,0],
  [1,0,1,0,0],
  [0,0,0,0,0]
]
```

Q4. **/1**

Donnez la liste d'adjacence du **Graphe 3** sous forme de dictionnaire Python.

```
G = {
  "A": ["B", "C"],
  "B": ["D", "E"],
  "C": [],
  "D": ["A", "C"],
  "E": []
}
```

Pour les fonctions suivantes, on considère que le graphe est représenté par une liste d'adjacence sous forme de dictionnaire Python.

Q5. **/1**

Écrire une fonction `est_successeur(G: dict, u: str, v: str) -> bool` qui vérifie si un sommet v est un successeur du sommet u dans G (donc si un arc `u` $\rightarrow$ `v` existe).

```python
def est_successeur(G: dict, u: str, v: str) -> bool:
  return v in G[u]
```

Q6. **/1**

Écrire une fonction `est_predecesseur(G: dict, u: str, v: str) -> bool` qui vérifie si un sommet v est un prédécesseur du sommet u dans G (donc si un arc `v` $\rightarrow$ `u` existe).

```python
def est_predecesseur(G: dict, u: str, v: str) -> bool:
  return u in G[v]
```

Q7. **/1**

Écrire une fonction `degres_sortant(G: dict, u: str) -> int` qui retourne le degré sortant du sommet u dans G.

```python
def degres_sortant(G: dict, u: str) -> int:
  return len(G[u])
```

Q8. **/2**

Écrire une fonction `degres_entrant(G: dict, u: str) -> int` qui retourne le degré entrant du sommet u dans G.

```python
def degres_entrant(G: dict, u: str) -> int:
  res = 0
  for v in G:
    if u in G[v]:
      res += 1
  return res
```

Complexité : $O(|S| + |A|)$ On parcourt tout les sommets et tout les arcs une fois.

3. Parcours

Q9. **/2**

Compléter la fonction `est_accessible(G: dict, u: str, v: str) -> bool` qui vérifie si un sommet v est accessible depuis le sommet u dans G. Vous implémenterez cette fonction sur une base du **parcours en profondeur**.

```
1  def est_accessible(G, src, dest):
2      # initialisation d'une pile et d'un ensemble de sommets visités
3      visites = set()
4      pile = [src]
5      while len(pile) != 0:
6          sommet = pile.pop()
7          if sommet not in visites:
8              visites.add(sommet)
9              for successeur in G[sommet]:
10                 if successeur == dest: # on a trouvé le sommet dest
11                     return True
12                 elif successeur not in visites:
13                     pile.append(successeur)
14     return False
```

Q10. **/3**

Écrire une fonction `taille_plus_court_chemin(G: dict, src: str, dest: str) -> int` qui retourne la taille du plus court chemin entre les sommets `src` et `dest` dans G.

Vous avez à disposition un programme de parcours en largeur. Reprenez le code en ajoutant les modifications nécessaires.

```python
def taille_plus_court_chemin(G, src, dest):
    visites = {src}
    file = [(src, 0)]

    while len(file) != 0:
        # On défile le premier élément ajouté
        sommet, dist = file.pop(0)

        # On ajoute les sucesseur non visités à la file
        for successeur in G[sommet]:
            # on s'arrête si on tombe sur notre distination
            if successeur == dest:
                return dist + 1
            if successeur not in visites:
                visites.add(successeur)
                file.append((successeur, dist+1))
    return None
```

4. Dijkstra

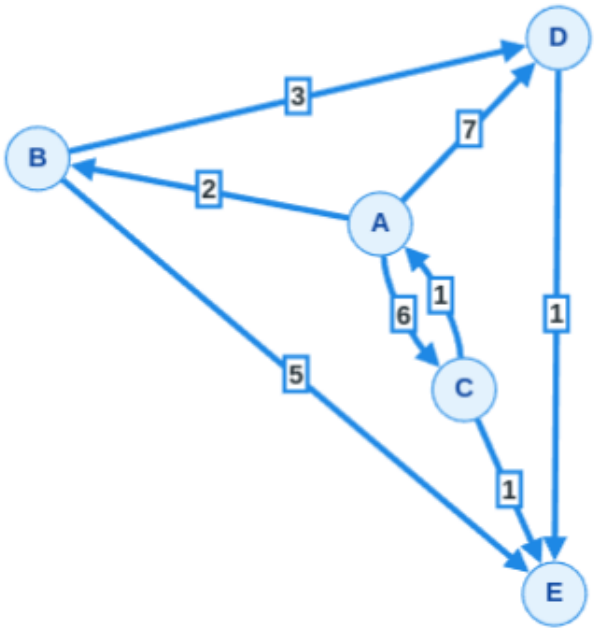

Fig. 4. – Graphe 4

Q11. **/2**

Dépilez l'algorithme de Dijkstra sur le **Graphe 4** en partant du sommet `A` en complétant le tableau suivant pour chaque tour de boucle.

Action	**A**	**B**	**C**	**D**	**E**	**`a_visiter`**
Initialisation	0	∞	∞	∞	∞	`{A, B, C, D, E}`
On regarde A (0)	0	2	6	7	∞	`{B, C, D, E}`
On regarde B (2)	0	2	6	5	7	`{C, D, E}`
On regarde D (5)	0	2	6	5	6	`{C, E}`
On regarde C (6)	0	2	6	5	6	`{E}`
On regarde E (6)	0	2	6	5	6	`{}`